KWAME NKRUMAH UNIVERSITY OF SCIENCE AND TECHNOLOGY
COLLEGE OF ENGINEERING

GROUP 6

GROUP MEMBERS:

1. ADU DAVID 7341223
2. ANANE KATRINA DUAMU 7342123
3. ABDUL AZIZ SUGLO 7344223
4. ASEMHIA PETER 7342523
5. OSEI PREMPEH MICHELLE 7343923
6. AMANKWAH ANDY KONADU 7341923

TABLE OF CONTENT

1. Introduction and Problem Statement..... 3
2. Methodology 4
3. Demonstration/Simulation:. 12
4. Final Conclusion..... 15

1. INTRODUCTION AND PROBLEM STATEMENT

1.1 Introduction

The **Smart Home Lighting System** is an automated solution designed to optimize residential electricity usage through intelligent sensing. Utilizing the **ESP32 microcontroller**, the system merges Passive Infrared (PIR) motion detection with manual serial overrides. This project serves as a foundational prototype for energy-efficient "Smart City" initiatives and automated building management systems.

1.2 Problem Statement

Traditional lighting relies entirely on human intervention. This leads to two primary issues:

Energy Waste: Lights often remain active in empty rooms, significantly increasing carbon footprints and utility costs.

Accessibility Constraints: Physical switches can be difficult to locate in total darkness or may be inconvenient for individuals with limited mobility.

The objective is to create a system that intelligently responds to human presence while allowing a user to maintain ultimate control through a digital interface.

2.METHODOLOGY

COMPONENT ANALYSIS

The hardware selection prioritizes low power consumption and high responsiveness. The following table details the primary hardware used in the circuit design:

Component	Specification/Role
ESP32 DevKit V1	A dual-core processor with integrated GPIOs used to execute the logic and manage timing.
PIR Sensor (HC-SR501)	Detects infrared heat signatures. It outputs a **HIGH** signal when motion is detected.
Red LED	Serves as the prototype "Light" actuator.
220Ω Resistor	A current-limiting resistor placed in series with the LED to protect the circuit (V = IR).
Connecting Wires	Jumper wires used to establish shared Ground and Signal paths.

3. CODE LOGIC (CORE FUNCTIONALITY)

The software follows an **Event-Driven Architecture**. Rather than stopping the program to wait for motion, the code uses non-blocking logic to monitor multiple inputs simultaneously.

3.1 Manual Serial Control

The system constantly polls the Serial.available() buffer. This allows the user to send commands from a computer keyboard:

Command '1': Forces the LED to a HIGH state (ON).

Command '0': Forces the LED to a LOW state (OFF).

Feedback: The system acknowledges every command via a serial print-out, ensuring the user is aware of the current system state.

Code Summary

1. Overview of the Loop Function

The loop() function runs continuously after the system starts. It controls the operation of the smart lighting system by:

Reading user commands from the Serial Monitor

Detecting motion using a PIR sensor

Automatically turning the light OFF after a set time

2. Serial Control (Manual Operation)

The system first checks if the user has entered a command through the Serial Monitor.

If **'1' is entered**:The LED light turns **ON**

The system records the current time using milliseconds

A confirmation message is displayed

If **'0' is entered**:

The LED light turns **OFF**

The system updates the light status

If any other key is pressed:

The system displays an **invalid command message**

This part allows **manual control** of the lighting system.

3. Motion Detection (Automatic Operation)

The system continuously reads the PIR sensor:

If motion is detected:

A message is displayed: *"Motion Detected"*

The LED turns **ON** (if not already ON)

The time of detection is recorded

It compares:

Current motion state

Previous motion state

This ensures motion is detected **only when it starts**, not continuously.

4. Auto-Off Timer

The system uses a timer to save energy:

If:

The light is ON

No motion is detected for a certain time (e.g., 5 seconds)

Then:

The LED turns **OFF automatically**

A message is displayed

This improves **energy efficiency**

5. System Stability

A small delay is included at the end of the loop

This prevents the system from running too fast and overwhelming the micro-controller

Final Conclusion

The code integrates **manual control, motion detection, and automatic timing** into one system. It ensures that:

The light responds to both user input and environmental changes

Energy is conserved through automatic shut-off

The system operates continuously and efficiently

```
// ============================================
// SMART HOME LIGHTING SYSTEM
// Motion Sensor (PIR) + Serial Control (1=ON, 0=OFF)
// ============================================

// Pin definitions
const int PIR_PIN = 33;     // PIR motion sensor connected to GPIO 33
const int LED_PIN = 14;     // LED connected to GPIO 14

// Variables for motion detection
int motionState = LOW;      // Current motion state
int lastMotionState = LOW;  // Previous motion state
bool lightOn = false;       // Light status
unsigned long lastMotionTime = 0;     // Time when last motion was detected
const unsigned long lightDuration = 5000;  // Light stays ON for 5 seconds

// Variables for serial control
char serialCommand = ' ';   // Store command from Serial Monitor

// ============================================
// SETUP - Runs once at the beginning
// ============================================
void setup() {
  // Start serial communication
  Serial.begin(115200);

  // Configure pins
  pinMode(PIR_PIN, INPUT);   // PIR sensor as INPUT
  pinMode(LED_PIN, OUTPUT);  // LED as OUTPUT

  // Make sure LED starts OFF
  digitalWrite(LED_PIN, LOW);

  // Print startup message
  Serial.println();
```

Serial Monitor

```
  Serial.println("==========================================");
  Serial.println("🏠 SMART HOME LIGHTING SYSTEM");
  Serial.println("==========================================");
  Serial.println("🔴 MOTION SENSOR: PIR (GPIO 33)");
  Serial.println("💡 LIGHT: LED (GPIO 14)");
  Serial.println("==========================================");
  Serial.println();
  Serial.println("🎮 CONTROL INSTRUCTIONS:");
  Serial.println("   Press 1 → Turn Light ON");
  Serial.println("   Press 0 → Turn Light OFF");
  Serial.println("==========================================");
  Serial.println();
  Serial.println("🎯 SYSTEM READY!");
  Serial.println();
  Serial.print("👉 Enter command (1 or 0): ");
}

// ==============================================
// LOOP - Runs repeatedly forever
// ==============================================
void loop() {
  // ==========================================
  // PART 1: CHECK SERIAL MONITOR FOR 1 or 0
  // ==========================================
  if (Serial.available() > 0) {
    // Read the incoming character
    serialCommand = Serial.read();

    // Check if '1' was pressed
    if (serialCommand == '1') {
      // Turn ON the light
      digitalWrite(LED_PIN, HIGH);
      lightOn = true;
      lastMotionTime = millis();  // Reset timer

Serial Monitor
      Serial.println();
      Serial.println("✅ COMMAND RECEIVED: 1");
      Serial.println("💡 Light TURNED ON");
      Serial.println("==========================================");
      Serial.println();
      Serial.print("👉 Enter command (1 or 0): ");
    }
    // Check if '0' was pressed
    else if (serialCommand == '0') {
      // Turn OFF the light
      digitalWrite(LED_PIN, LOW);
      lightOn = false;

      Serial.println();
      Serial.println("✅ COMMAND RECEIVED: 0");
      Serial.println("💡 Light TURNED OFF");
      Serial.println("==========================================");
      Serial.println();
      Serial.print("👉 Enter command (1 or 0): ");
    }
    // If any other key is pressed
    else if (serialCommand != '\n' && serialCommand != '\r') {
      Serial.println();
      Serial.print("❌ Invalid command: ");
      Serial.print(serialCommand);
      Serial.println(" - Please press 1 or 0 only");
      Serial.print("👉 Enter command (1 or 0): ");
    }
  }

  // ==========================================
  // PART 2: CHECK FOR MOTION (PIR Sensor)
  // ==========================================
  // Read the current state of the PIR sensor
  motionState = digitalRead(PIR_PIN);
Serial Monitor
```

```
  // Read the current state of the PIR sensor
  motionState = digitalRead(PIR_PIN);

  // Check if motion is detected (HIGH means motion detected)
  if (motionState == HIGH && lastMotionState == LOW) {
    // Motion detected!
    Serial.println();
    Serial.println("========================================");
    Serial.println("🔴🔴🔴 MOTION DETECTED! 🔴🔴🔴");
    Serial.println("========================================");

    // Turn on the LED if it's not already on
    if (!lightOn) {
      digitalWrite(LED_PIN, HIGH);
      lightOn = true;
      Serial.println("💡 Light turned ON by motion sensor");
    } else {
      Serial.println("💡 Light was already ON");
    }

    // Record the time when motion was detected
    lastMotionTime = millis();

    Serial.println("========================================");
    Serial.println();
    Serial.print("👉 Enter command (1 or 0): ");
  }

  // Update last motion state
  lastMotionState = motionState;

  // ========================================
  // PART 3: AUTO-OFF TIMER
  // ========================================
  // Check if LED is on and no motion for longer than lightDuration
```

Serial Monitor

```
  // Check if LED is on and no motion for longer than lightDuration
  if (lightOn == true && (millis() - lastMotionTime) > lightDuration) {
    // Turn off the LED
    digitalWrite(LED_PIN, LOW);
    lightOn = false;

    Serial.println();
    Serial.println("⏰ Auto-off: No motion detected for 5 seconds");
    Serial.println("💡 Light turned OFF automatically");
    Serial.println("========================================");
    Serial.println();
    Serial.print("👉 Enter command (1 or 0): ");
  }

  // Small delay to prevent overwhelming the system
  delay(100);
}
```

Serial Monitor

Motion Detection Logic

The system uses **State Change Detection**. It does not just check if motion is happening; it checks if motion *just started*. If the sensor moves from LOW to HIGH, the "Light On" sequence is triggered.

The Auto-Off Timer

To prevent the light from staying on indefinitely, a timer is implemented using the millis() function. This allows the system to calculate:

Elapsed-Time = Current Time - Last Motion Time
If this value exceeds **5,000 milliseconds** (5 seconds), the system automatically cuts power to the LED.

5. DEMONSTRATION AND SIMULATION

5.1 Simulation Environment

The system was validated using the **Wokwi Simulation Platform**. This allowed for real-time testing of the code logic and virtual wiring without the risk of hardware damage.

5.2 Test Scenarios & Results

Motion Trigger: During testing, triggering the PIR sensor resulted in an immediate LED activation. The serial monitor correctly output: MOTION DETECTED!.

CIRCUIT DESIGN AND CONNECTIONS

The circuit is wired to ensure signal integrity and protection of the ESP32's internal circuitry.

Connection Map

PIR Sensor: The VCC is connected to the ESP32's **VIN** (5V). The Ground is shared with the ESP32. The output (D) pin is mapped to **GPIO 33**.

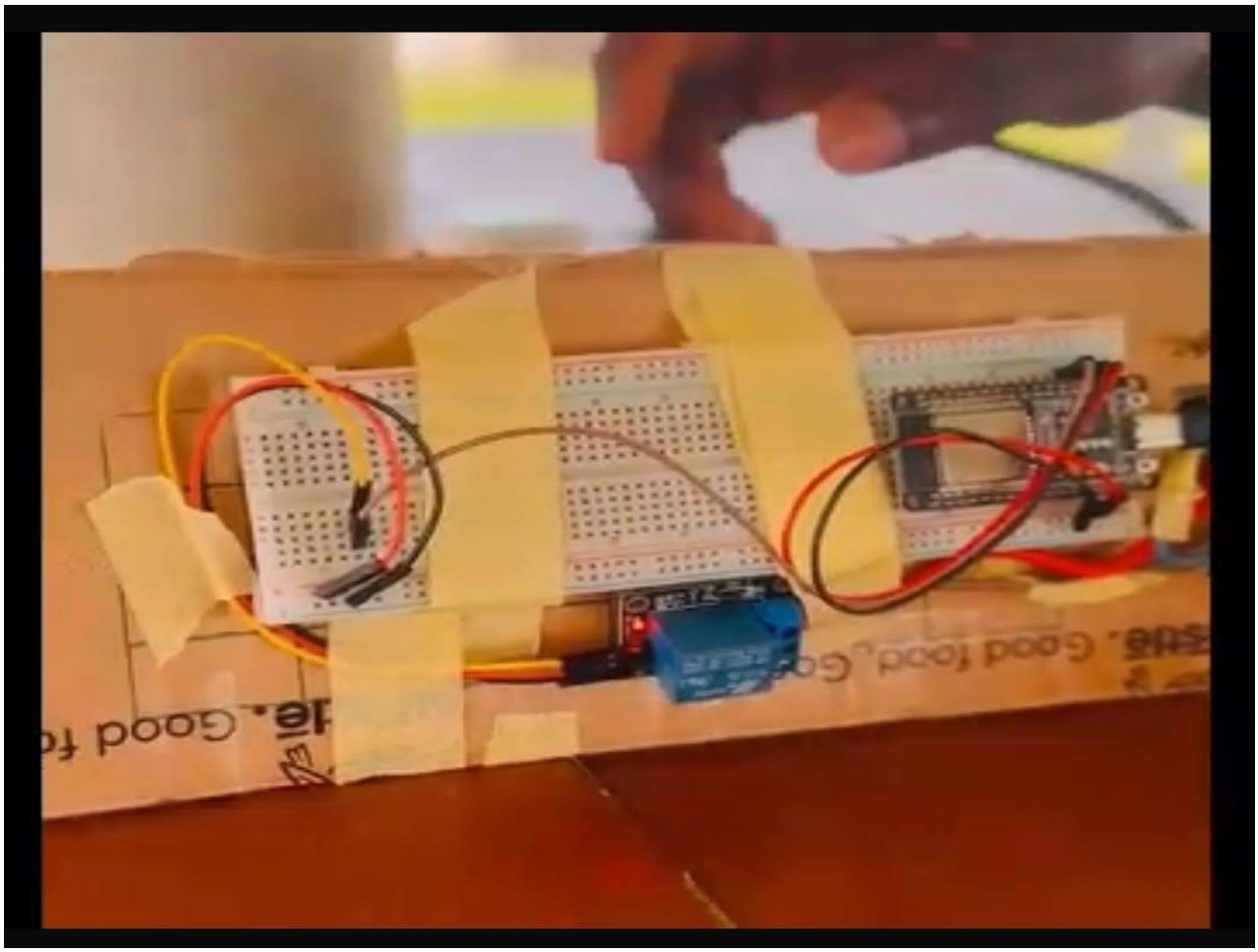
LED Output: The LED's long leg (Anode) is connected to **GPIO 14** via a **220Ω resistor**. The short leg (Cathode) is connected to **GND**.

Signal Flow

Input: PIR Sensor detects infrared movement → Sends 3.3V signal to GPIO 33.

Process: ESP32 identifies the signal, prints a "Motion Detected" message, and resets the activity timer.

Output: ESP32 sends 3.3V to GPIO 14 → LED illuminates.



Manual Override: Typing '1' or '0' successfully bypassed the sensor. This is a critical safety feature, allowing the user to turn the light off manually even if they are still moving in the room.

Timeout Accuracy: The 5-second auto-off function responded accurately, turning off the LED precisely when the inactivity threshold was met.

6. CONCLUSION

The Smart Home Lighting System effectively addresses the problem of energy inefficiency. By combining the **PIR sensor's** environmental awareness with the **ESP32's** processing power, we have created a "set-and-forget" system.

This project demonstrates that significant energy savings can be achieved through simple, low-cost automation logic. Furthermore, the use of the ESP32 provides a scalable model; in future iterations, this system could be connected to Wi-Fi to allow for smartphone notifications and remote energy monitoring.